

ICU Ventilator Triage under Scarcity

A Two-Layer Hybrid System for Allocation and Ethical Audit

Final Report for the Computational Ethics Course 2025/2026

Iago López Lamas
iago.lopezlamas@studio.unibo.it

June 22, 2026

During the COVID-19 health emergency, hospitals in most Western European countries had to make a series of difficult and ethically contested decisions on how to share scarce medical resources, in particular mechanical ventilators in overcrowded ICUs. In this project I look at that kind of scarcity scenario and design a decision-support system inspired by it, using ideas from the Computational Ethics course at the University of Bologna. The system is built in two separate layers. The first layer uses ASP/Clingo and is in charge of choosing which patients receive a ventilator. The second layer uses the Arg2P argumentation framework and works as an independent ethical check on that choice, deciding whether the allocation can be defended from a moral point of view. The main idea behind the project is that these two layers should stay independent, so that the second one is not just an explanation of the first, but a real second opinion that is allowed to disagree.

1 Introduction

The COVID-19 pandemic pushed many European hospitals into a situation that medical ethics had mostly treated as a theoretical problem until then: there were more patients in need of mechanical ventilation than ventilators available. Doctors had to decide, often in a few minutes and without clear legal guidance, which patients would receive the device and which would not. National medical associations published triage protocols during the first weeks of the emergency, but these documents were written quickly and were criticised for mixing clinical criteria with implicit ethical assumptions that were never made explicit, such as giving priority to younger patients or to healthcare workers.

The starting point of this project is the observation that, in a real scarcity scenario, two very different questions get answered at the same time and by the same person: *who should get the ventilator* (an operational question) and *is this allocation morally defensible* (an ethical question). Mixing both is convenient under time pressure, but it

hides the moral reasoning inside a clinical decision and makes it almost impossible to audit afterwards.

This project explores what happens if those two questions are answered by two separate components. The first component, which I call Layer 1, is an allocator written in ASP/Clingo: given a set of patients and a number of available ventilators, it computes an allocation by maximising a lexicographic objective (lives, then life-years, then quality-adjusted life-years), subject to a hard floor of clinical eligibility based on the IDSA criteria. Ties are broken by an external lottery with a fixed random seed, so that the result is always reproducible. The second component, Layer 2, is an ethical auditor built on the Arg2P argumentation framework. It reads the allocation produced by Layer 1 together with the original case and evaluates it against Ross’s *prima facie* duties (non-maleficence, beneficence, fidelity, justice, gratitude), returning one of three possible verdicts: *defensible*, *indefensible* or *undecided* under grounded semantics.

The main contribution is not the allocator itself, which is a fairly standard piece of optimisation, but the architectural choice of keeping the two layers independent. Layer 2 is not a justification of Layer 1: it is allowed to disagree, and the *undecided* verdict is treated as a valid answer rather than as a failure of the system, because some moral conflicts are genuinely irresolvable and a system that always produces a clean answer would misrepresent the ethics of the situation.

2 Ethical Framework and Tools

2.1 Ethical background

Consequentialism judges an action only by its outcomes: the right allocation is the one that produces the best aggregate result, typically measured in lives saved or in quality-adjusted life-years (QALYs).

Deontology, in the Kantian tradition, judges an action by whether it respects a set of duties or rights, regardless of the outcome.

A pure version of either theory is hard to defend in a triage context: pure consequentialism allows actions that most people would find unacceptable (such as openly discriminating against a disabled patient if it raises the QALY total), while pure deontology gives little guidance when duties conflict and a decision must still be made.

This project combines both approaches. Hard constraints first remove the forbidden allocations (clinically ineligible patients, or rankings based on protected attributes such as disability), and only then does the solver optimise the objective on what is left. The deontological rules therefore act as a floor that the optimisation is not allowed to cross, while the consequentialist component decides among the allocations that remain.

For the ethical audit, the project uses W.D. Ross’s theory of *prima facie* duties. Ross argues that moral life is governed by several duties that are all binding in principle, but that can be overridden by other duties in a concrete situation. None of them is absolute. The five duties used in this project are summarised in Table 1.

Duty	Meaning in the triage context
Non-maleficence	Do not actively cause harm to a patient
Beneficence	Act to produce the greatest medical good
Fidelity	Keep prior commitments to patients already under treatment
Justice	Distribute resources fairly, without arbitrary discrimination
Gratitude	Recognise prior service, e.g. that of healthcare workers

Table 1: The five prima facie duties used by Layer 2.

2.2 Computational tools

Layer 1 is written in *Answer Set Programming* (ASP), a declarative logic-programming paradigm in which a problem is described as a set of rules and constraints, and a solver (in this case Clingo) returns the models that satisfy them. ASP is well suited to combinatorial problems with hard constraints and a clear optimisation criterion, which matches the structure of a triage allocation.

Layer 2 is written in *Arg2P-KT*, a Kotlin implementation of a defeasible argumentation framework. In this kind of system, rules are not necessarily true: they can attack each other, and the final status of an argument (accepted, rejected or undecided) is computed under a chosen semantics. This conservative choice is what allows the system to return *undecided* when duties genuinely conflict.

3 System Architecture

The system has two layers that run in sequence and communicate through a JSON file. Layer 1 takes the description of a triage case and produces an allocation; Layer 2 reads that allocation and returns an ethical verdict.

3.1 Layer 1 – Allocator (ASP/Clingo)

A case is described by a set of patients, a set of available ventilators, and a list of clinical and contextual facts per patient (age, SOFA score, Charlson Comorbidity Index, survival probability, life expectancy, quality weight, and a few flags such as `healthcare_worker` or `disability_permanent`). The allocator does not use age, SOFA or CCI as allocation criteria; SOFA and CCI are absorbed into the precomputed `idsa_eligible` flag.

The allocator is governed by five hard constraints, summarised in Table 2. They act as the deontological floor and remove any allocation that violates them before the optimisation starts.

Id	Constraint
H1	Only IDSA-eligible patients can be assigned a ventilator
H2	A ventilator can be used only if it is available
H3	At most one ventilator per patient, at most one patient per ventilator
H4	No ventilator may stay idle if an eligible patient is unassigned
H5	The vocabulary contains no sex, race, religion or orientation predicates

Table 2: Hard constraints enforced by Layer 1.

On top of this floor, the allocator maximises an objective with three levels: 1st) the number of effective lives saved; 2nd) the expected life-years; and 3rd) the expected QALYs. The score `effective_lives_contribution` is normally 1 per patient, but a fact `instrumental_lives_factor(P, N)` with $N > 1$ raises it; this is how the rule-utilitarian argument for healthcare workers is encoded, and it is made explicit in Layer 1 precisely so that Layer 2 can later audit it (Case 05).

When the three levels of the objective still produce more than one optimal allocation, Clingo emits all of them and a short Python wrapper (`run.sh`) chooses one uniformly at random using a seed read from the case file. Storing the seed in the case file makes every run reproducible, which is important both for testing and for the audit.

3.2 Layer 2 – Ethical auditor (Arg2P-KT)

Layer 2 receives the allocation produced by Layer 1, together with the original case, and decides whether the allocation can be morally defended. It is built on top of Arg2P-KT, but two short Python steps wrap the framework. The first step pre-computes a small set of ground flags that the argumentation rules need but that cannot be expressed inside Arg2P itself, such as whether the QALY level was decisive for excluding a patient (`qaly_decisive`), whether there was more than one optimal allocation

(`tied_optimal_gt1`), or whether a patient with an instrumental factor was assigned. The second step injects these flags as facts and runs the framework.

The argumentative theory consists of ten defeasible rules, organised by Rossian duty (Table 3). Rules of the form r_X build arguments for or against the allocation; an additional set of bridge rules encodes the attacks between them (for example, a rule that derives *discriminatory_exclusion* attacks any rule that claims the allocation is beneficent). Two preferences are declared with `sup/2`, both saying that non-maleficence (NM2) and justice (J1) override beneficence (B1) in discrimination cases.

Rule	Duty	Conclusion when the body fires
F1	Fidelity	The allocation violates fidelity
NM1	Non-maleficence	Withdrawal counts as active harm
NM2	Non-maleficence	The exclusion is discriminatory
J1	Justice	The exclusion is unjust
J2	Justice	The lottery satisfies procedural fairness
J3	Justice	Equal consideration is violated
B1	Beneficence	The allocation serves individual beneficence
B2	Beneficence	The allocation serves aggregate beneficence
B3	Beneficence	A third party (fetus) is left unattended
G1	Gratitude	The patient deserves gratitude

Table 3: Defeasible rules of Layer 2, grouped by Rossian duty.

The framework returns one of three verdicts. An allocation is reported as defensible if the main argument supporting it survives every attack, indefensible if it is defeated by an argument that survives in turn, and undecided if no argument in the relevant chain can be conclusively accepted or rejected. The third outcome is not a bug: it is the honest answer when two *prima facie* duties conflict and neither has enough weight to defeat the other.

4 Canonical Cases

To test if the two-layer architecture behaves as expected, I designed six ethical tension cases. The cases are deliberately small (two to five patients, one to three ventilators) so that the allocation produced by Layer 1 is easy to verify by hand and the audit produced by Layer 2 can be traced rule by rule. Table 4 summarises them.

#	Case	Ethical tension
01	withdrawal	Act/omission asymmetry, fidelity
02	pregnancy	Duty toward a third party (fetus)
03	disability	QALY weight as discrimination
04	clinical_tie	Pure tie, fairness of the lottery
05	healthcare_worker	Rule-utilitarian boost vs. equal consideration
06	extreme_scarcity	Scale of denied care

Table 4: The six canonical cases and the ethical tension each one isolates.

Case 01 – Withdrawal. Patient `p1` is already on a ventilator when a new patient `p3` arrives with clearly better expected outcomes. Layer 1 withdraws `p1` to reassign the device.

Case 02 – Pregnancy. A pregnant patient at 28 gestational weeks is one of three eligible candidates and has marginally worse expected outcomes than the other two. Layer 1 excludes her.

Case 03 – Disability. Two patients are clinically identical (same SOFA, same survival probability, same life expectancy), but one has a permanent disability that lowers her QALY quality weight. The QALY level becomes the only discriminator.

Case 04 – Clinical tie. Three patients are symmetric on every clinical and contextual variable. The hierarchy cannot distinguish them and the lottery has to step in.

Case 05 – Healthcare worker. A patient who is a doctor has worse individual outcomes than another candidate, but is given an `instrumental_lives_factor` of 5 in the case file. Layer 1 assigns the ventilator to her.

Case 06 – Extreme scarcity. Five eligible patients compete for a single ventilator. Layer 1 picks the one with the best individual outcome and leaves four patients without the device.

5 Results

Table 5 summarises the outcome: the patient(s) chosen by Layer 1, the number of optimal models tied at the lexicographic top, the final verdict returned by Layer 2, and the defeasible rules whose conclusion ended up labelled **in** on each side of the dispute.

#	Case	Allocation	Ties	Verdict	Pro in	Contra in
01	withdrawal	{p2, p3}, withdraw p1	2	undecided	J2	F1, NM1
02	pregnancy	{p1, p3}	2	undecided	J2	B3
03	disability	{p2}	1	indefensible	—	J1, NM2
04	clinical_tie	{p1, p3} (lottery)	6	defensible	B1, J2	—
05	healthcare_worker	{p1}	1	undecided	—	J3
06	extreme_scarcity	{p1}	1	defensible	B1	—

Table 5: Results per case layers 1 and 2

The verdicts span all three possible categories. Case 04 returns *defensible* because the lottery is the only way to break a perfectly symmetric tie and J2 (procedural fairness) is supported with no rule attacking it. Case 06 also returns *defensible*, but for a different reason that is examined separately below. Case 03 is the only *indefensible*: with two clinically identical patients, the QALY weight is the sole reason for excluding the disabled one, which makes NM2 and J1 fire and the preferences declared with **sup**/2 silence B1. Cases 01, 02 and 05 return *undecided*, each for a different reason: a withdrawal that violates fidelity but improves outcomes, a third-party duty toward the fetus that the model cannot fully resolve, and a rule-utilitarian boost for a healthcare worker that B2 and G1 support but J3 attacks without a declared preference on either side.

Case 06 is the most informative one. Four patients out of five are denied care, and yet Layer 2 returns *defensible*: it sees that the patient with the best individual outcome was assigned, B1 fires, and no rule in the current theory detects the scale of denied care. The verdict is internally consistent with the rule set, but the rule set itself is arguably incomplete on this point. This limitation is discussed in the next section.

6 Conclusion and Future Work

This project has presented a two-layer hybrid system for ICU ventilator triage under scarcity. Layer 1 produces an allocation by maximising a objective subject to a hard deontological floor, and Layer 2 audits that allocation independently against Ross’s prima facie duties under grounded semantics.

As a possible evolution of the project, an interesting direction would be to redesign the system around three independent engines that run in parallel and then converge on a final decision. Engine U (utilitarian) would be an ASP/Clingo solver maximising an aggregate objective, in the spirit of the current Layer 1. Engine D (deontological) would be a separate module encoding strict Kantian rules as inviolable constraints, without any optimisation component. Engine R (Rossian) would be an Arg2P module that reasons over prima facie duties as in the current Layer 2. Each engine would produce its own verdict from its own ethical tradition, and a final stage would combine the three into a single decision.